

UNIVERSIDAD NACIONAL AUTÓNOMA
DE MÉXICO

FACULTAD DE CIENCIAS

MODELADO Y PROGRAMACIÓN

CASOS PRUEBA

PRÁCTICA 4:
PRUEBAS UNITARIAS

DISEÑO DE PRUEBAS PARA UN SISTEMA

P R E S E N T A N :

INTEGRANTES DEL EQUIPO:

Alan Gael Alvarez Hernández
María Fernanda Miranda Sánchez
Pedro Pablo Ruiz Hernández

PROFESORA:
Karla Adriana Esquivel Guzmán

AYUDANTE DE TEORÍA:
Ossiel Aguilar Espíndola

AYUDANTE DE LABORATORIO:
Enoc Pineda Aguirre



FACULTAD DE CIENCIAS
UNAM

SEMESTRE 1-2027

Ciudad de México, 20 de octubre de 2026

Índice

1. Código original **3**

2. Reglas del Sistema: Ticket de Estacionamiento **4**

3. Construcción de la Tabla de casos **4**

3.1. Planteamiento de pruebas 4

3.2. Tabla de casos 5

4. Evidencia de Ejecución **6**

4.1. Casos Normales: 1, 2 y 3 6

4.2. Casos Frontera: 4, 5, 6, 7, 8 y 9 7

4.3. Casos Entrada Inválida: 10, 11 y 15 8

4.4. Casos Interacción entre reglas: 12 y 13 10

4.5. Caso Números decimales en el resultado: 14 10

5. Preguntas de análisis **10**

Código original

Código original a evaluar con una suit de pruebas, estas últimas deben sacarse del modelo del Sistema de Ticket estacionamiento, no del Código original.

```
# Tarifa del boleto perdido.
TARIFA_BOLETO_PERDIDO = 300.0

def calcular_total(self, minutos: int, tipo_cliente: str = "normal",
                   boleto_perdido: bool = False) -> float:
    """
    Calcula el total a pagar de acuerdo con el modelo de la práctica.

    Nota para el alumno:
    trate esta implementación como un sistema que debe verificarse.
    No asuma que todo lo que hace el código es correcto.
    """
    # Entrada Invalida: Minutos Negativos.
    if minutos < 0:
        raise ValueError("Los minutos no pueden ser negativos")

    # Entrada Inválida: Tipo de cliente no reconocido.
    if tipo_cliente not in ("normal", "frecuente"):
        raise ValueError("Tipo de cliente no válido")

    # Si el boleto está perdido -> $300.00
    if boleto_perdido:
        total = self.TARIFA_BOLETO_PERDIDO

    # Si permanece 15 minutos o menos -> $0.00
    elif minutos < 15:
        total = 0.0

    # Si permanece 60 minutos o menos -> $20.00
    elif minutos <= 60:
        total = 20.0

    # Si permanece mas de 60 minutos:
    # -> $20.00 la primera hora.
    # -> $15.00 cada hora iniciada.
    else:
        horas_adicionales = (minutos - 60) // 60
        total = 20.0 + (horas_adicionales * 15.0)

    # Si el cliente es "frecuente" aplicar el 10% de descuento.
    if tipo_cliente == "frecuente":
        total *= 0.90

    # Devolver el total redondeado con dos decimales.
    return round(total, 2)
```

Reglas del Sistema: Ticket de Estacionamiento

Las reglas del Sistema de Ticket de Estacionamiento son:

R1. Los minutos deben ser un número entero mayor o igual que 0.

R2. Los tipos de clientes válidos son:

- **"normal"**
- **"frecuente"**

R3. Si el boleto está perdido, el costo final es de $300.00, sin importar los minutos ni el tipo de cliente.

R4. Si los minutos son 0, el costo es $0.00

R5. De 1 a 15 minutos, el costo es $0.00

R6. De 16 a 60 minutos, el costo es de $20.00

R7. Después de 60 minutos se cobran $20.00 por la primera hora y $15.00 por cada hora adicional **iniciada**.

R8. Un cliente **"frecuente"** recibe un 10 % de descuento sobre el costo calculado, excepto cuando perdió el boleto.

R9. El costo nunca puede ser negativo.

R10. Si los minutos son negativos, el sistema debe lanzar un error.

R11. Si el tipo de cliente no es válido, el sistema debe lanzar un error.

Construcción de la Tabla de casos

Planteamiento de pruebas

Planteamos para el Sistema de Ticket de Estacionamiento:

- 3 casos normales;
- 6 casos frontera;
- 2 entradas inválidas;
- 2 casos de interacción entre reglas;
- 1 caso para números decimales en el resultado;
- 1 caso adicional propuesto por el equipo (entrada inválida).

Tabla de casos

Núm.	Caso	Entrada	Resultado esperado	Regla(s)	Justificación
1	Normal	(10 minutos, "normal", con boleto)	$0.00	R5	Verificamos que se calcule correctamente el cobro a una persona normal con boleto y 10 minutos.
2	Normal	(40 minutos, "normal", con boleto)	$20.00	R6	Verificamos que se calcule correctamente el cobro a una persona normal con boleto y 40 minutos.
3	Normal	(1250 minutos, "normal", con boleto)	$320.00	R7	Verificamos que se calcule correctamente el cobro a una persona normal con boleto y 1250 minutos.
4	Frontera	(15, "normal", con boleto)	$0.00	R5	Verificamos que haya un cambio en la frontera entre 15 y 16.
5	Frontera	(16, "normal", con boleto)	$20.00	R6	Verificamos que haya un cambio en la frontera entre 15 y 16.
6	Frontera	(60, "normal", con boleto)	$20.00	R6	Verificamos que haya un cambio en la frontera entre 60 y 61.
7	Frontera	(61, "normal", con boleto)	$35.00	R7	Verificamos que haya un cambio en la frontera entre 60 y 61.
8	Frontera	(120, "normal", con boleto)	$35.00	R7	Verificamos que haya un cambio en la frontera entre 120 y 121.
9	Frontera	(121, "normal", con boleto)	$50.00	R7	Verificamos que haya un cambio en la frontera entre 120 y 121.
10	Entrada inválida	(-1, "normal", con boleto)	Lanza Excepción: Minutos inválidos.	R1 y R10	Verificamos que el sistema no acepte minutos negativos.
11	Entrada inválida	(5, "premium", con boleto)	Lanza Excepción: Tipo de cliente inválido.	R2 y R11	Verificamos que sólo acepte los clientes válidos.

Núm.	Caso	Entrada	Resultado esperado	Regla(s)	Justificación
12	Interacción entre reglas	(1250, "frecuente", con boleto)	$288.00	R7 y R8	Verificamos que al cliente "frecuente" con boleto se le aplique su correspondiente descuento dados los minutos que pasó en el estacionamiento.
13	Interacción entre reglas	(1250, "frecuente", sin boleto)	$300.00	R3 y R8	Aunque el cliente sea "frecuente", si perdió el boleto no se aplica el descuento y el costo final es de $300.00.
14	Números decimales en el resultado	(61, "frecuente", con boleto)	$31.50	R7 y R8	Verificamos que el descuento con decimales se aplique adecuadamente al cliente "frecuente".
15	Entrada inválida	(15.5, "normal", con boleto)	Lanza Excepción: Minutos inválidos.	R1	El sistema solo permite minutos enteros; no acepta valores decimales.

Tabla 1: Casos establecidos para el Sistema: Ticket de Estacionamiento.

Evidencia de Ejecución

Casos Normales: 1, 2 y 3

```
# Arrange
@pytest.mark.parametrize(
    "minutos, esperado",
    [
                        # | Caso | Descripción                                |
        (10, 0.00),     # |  1   | Cliente "normal" con boleto pasó 10 min.   |
        (40, 20.00),    # |  2   | Cliente "normal" con boleto pasó 40 min.   |
        (1250, 320.00), # |  3   | Cliente "normal" con boleto pasó 1250 min. |
    ],
)
def test_cliente_Normal_con_Boleto(sistema, minutos, esperado):
    """
    Verifica el cobro calculado para un cliente "normal" con boleto
    dado un cierto tiempo en minutos.
    """

    # Act
    resultado = sistema.calcular_total(minutos, "normal", False)

    # Assert
    assert resultado == pytest.approx(esperado)
```

Al ejecutar las pruebas sin modificar el código original, falla el Caso 3 establecido en las pruebas; pero como comprobamos el resultado a obtener antes de ejecutar la prueba, concluimos que el código original falló la prueba y debemos corregirlo.

```
...
    # Si permanece mas de 60 minutos:
    # -> $20.00 la primera hora.
    # -> $15.00 cada hora iniciada.
    else:
        horas_adicionales = (minutos - 60) // 60
        total = 20.0 + (horas_adicionales * 15.0)
...
```

Al observar el código original, debemos modificar el cálculo si el cliente permanece mas de 60 minutos, pues al dividir las horas adicionales, solo se toman en cuenta las horas completas que ya pasaron, pero falta contemplar las horas iniciadas, para compensar, debemos tomar el techo del resultado de la división:

```
...
    # Si permanece mas de 60 minutos:
    # -> $20.00 la primera hora.
    # -> $15.00 cada hora iniciada.
    else:
        horas_adicionales = math.ceil((minutos - 60) / 60) # Modificación
        total = 20.0 + (horas_adicionales * 15.0)
...
```

Con este cambio, solucionamos un problema del código original y pasan las primeras tres pruebas de los primeros tres casos.

Casos Frontera: 4, 5, 6, 7, 8 y 9

Hacemos las pruebas con clientes de tipo "normal" y con boleto.

```
# Arrange
@pytest.mark.parametrize(
    "minutos, esperado",
    [
                      # | Caso | Descripción                                          |
        (15, 0.00),   # |  4   | Último valor entre el rango de 1-15.                 |
        (16, 20.00),  # |  5   | Primer valor entre el rango de 16-60.                |
        (60, 20.00),  # |  6   | Último valor entre el rango de 16-60.                |
        (61, 35.00),  # |  7   | Una hora completa y una hora adicional iniciada.     |
        (120, 35.00), # |  8   | Dos horas completas sin hora adicional iniciada.     |
        (121, 50.00), # |  9   | Dos horas completas y una hora adicional iniciada.   |
    ],
)
```

```
def test_cliente_Normal_con_Boleto_Casos_Frontera(sistema, minutos, esperado):
    """
    Verifica el cobro calculado para un cliente "normal" con boleto dado un cierto tiempo,
    en escencia cuando hay un cambio entre los límites de rangos de tiempo.
    """

    # Act
    resultado = sistema.calcular_total(minutos, "normal", False)

    # Assert
    assert resultado == pytest.approx(esperado)
```

Al ejecutar las pruebas sin modificar el código original, falla el Caso 4 establecido en las pruebas; pero como comprobamos el resultado a obtener antes de ejecutar la prueba, concluimos que el código original falló la prueba y debemos corregirlo.

```
...
    # Si permanece 15 minutos o menos -> $0.00
    elif minutos < 15:
        total = 0.0
...
```

Al observar el código original, notamos que debemos modificar el cálculo en la frontera de 15 minutos, pues no toma en cuenta los 15 minutos para asignar el costo correcto, para solucionarlo, debemos agregar el valor de 15 minutos en la frontera:

```
...
    # Si permanece 15 minutos o menos -> $0.00
    elif minutos <= 15: # Modificación
        total = 0.0
...
```

Con este cambio, solucionamos otro problema del código original y pasan las pruebas de los casos 4, 5, 6, 7, 8 y 9.

Casos Entrada Inválida: 10, 11 y 15

Hacemos las pruebas con un cliente de tipo "normal" y con boleto para el caso 10 y 15; para el caso 11 un cliente con boleto y 5 minutos en el estacionamiento.

Dado que no tenemos excepciones personalizadas en el Código orginal, nosotros definimos las Excepciones que se deben lanzar al detectar una Entrada inválida:

- **MinutosInvalidos:** Lanzada cuando no se cumple **R1** y **R10**.
- **ClienteInvalido:** Lanzada cuando no se cumple **R2**.

Recordemos que definir una misma Excepción (como ValueError) para los "errores" que lanze el programa es una mala práctica de programación, pues dificulta el mantenimiento del programa a largo plazo; por ejemplo, cuando se incremente la cantidad de código es dificil localizar en que parte del Código se encuentra la Excepción que debemos mejorar o arreglar.

```
...
from MinutosInvalidos import MinutosInvalidos
from ClienteInvalido import ClienteInvalido
...
# Test de Entradas inválidas.

# Arrange
@pytest.mark.parametrize(
    "minutos",
    [
                # | Caso | Descripción                      |
        -1,     # | 10   | -1 min no está bien definido.   |
        15.5,   # | 15   | 15.5 min no está bien definido. |
    ],
)
def test_Minutos_Invalidos(sistema, minutos):
    """
    Verifica que el programa rechaze los Minutos Inválidos establecidos
    en la R1.
    """
    # Siguiendo la R10, verifcamos que rechaze los minutos negativos.
    # Act and Assert
    with pytest.raises(MinutosInvalidos):
        sistema.calcular_total(minutos, "normal", False)

# Arrange
@pytest.mark.parametrize(
    "cliente",
    [
                    # | Caso | Descripción                                  |
        "premium",  # | 11   | Cliente "premium" no está bien definido.   |
    ],
)
def test_Cliente_Invalido(sistema, cliente):
    """
    Verifica que el programa rechaze los clientes que no están
    definidos en la R2. Siguiendo el procedimiento de R11.
    """

    # Act and Assert
    with pytest.raises(ClienteInvalido):
        sistema.calcular_total(5, cliente, False)
```

Al correr las pruebas, en el caso 10 y 11 no se detecta la excepción que esperabamos, esto debido a que el Código original tiene las Excepciones marcadas con la misma "etiqueta" de ValueError, por otro lado, en el caso 15 el código no lanzó la Excepción que esperabamos, y dado que hemos planteado bien los ressultados esperados por las pruebas, debemos mejorar el Código Original.

```
...
    # Entrada Invalida: Minutos Negativos.
    if minutos < 0:
        raise ValueError("Los minutos no pueden ser negativos")

    # Entrada Inválida: Tipo de cliente no reconocido.
    if tipo_cliente not in ("normal", "frecuente"):
       raise ValueError("Tipo de cliente no válido")
...
```

Notemos que en los casos 10 y 11, debemos ocupar las Excepciones que personalizamos para "etiquetar" de una mejor manera los casos de entradas inválidas, mientras que para el caso 15, el definido por el equipo, debemos agregar una Excepcion de MinutosInvalidos para que rechace minutos que no sean numeros enteros positivos o el cero, en este caso, decimales.

```
...
    # Entrada Invalida: Minutos Negativos.
    if minutos < 0:
        raise MinutosInvalidos("Los minutos no pueden ser negativos") # Modificacion

    # Entrada Invalida: Minutos Decimales. # Modificacion
    if isinstance(minutos, float):
        raise MinutosInvalidos("Los minutos no pueden ser en decimal")

    # Entrada Inválida: Tipo de cliente no reconocido.
    if tipo_cliente not in ("normal", "frecuente"):
        raise ClienteInvalido("Tipo de cliente no válido") # Modificacion
...
```

Con ello, hemos corregido errores del Código original y las pruebas 10, 11 y 15 pasan.

Casos Interacción entre reglas: 12 y 13

D12 y D13

Caso Números decimales en el resultado: 14

D14

Preguntas de análisis

1. ¿Qué diferencia hay entre un caso normal y un caso frontera?

R

2. ¿Cuál de sus pruebas considera más importante y por qué?

R

3. ¿Encontró algún comportamiento de la implementación que no coincida con el modelo?

R

4. ¿Una suite con 100 % de pruebas aprobadas demuestra que el programa es correcto? Explique.

R

5. Si una IA generara automáticamente 50 pruebas, ¿qué tendría que revisar una persona antes de confiar en ellas?

R